

JHU Biology REU 2026: Introduction to R

Michael Tassia

2026-05-28

R is a common object-oriented programming language for performing data analysis in biology. While other programming languages are also common in biology (e.g., python), R (and by extension, RStudio/Posit) is a common & flexible environment to simulate, analyze, and visualize biological data.

Why program? Programming presents a powerful way to interact and procedurally analyze/generate data in a reproducible manner (both in biology and other data-driven fields). In many subfields of biology (e.g., genetics and evolutionary biology), computer science has becoming inseparably intertwined with research. Becoming familiar/comfortable with one or two programming languages will provide far more flexibility when methods need to be catered to *your* data!

The goal of this 4-day workshop is to introduce the fundamentals of using R to analyze biological data. We'll start here with the basics of how to interact with the R programming environment and build off this foundation in the following lessons.

R and its libraries are frequently being updated, and new tools are being published every day to assist in data analysis. In this course, we'll only scratch the surface of computational biology. If you'd like further instruction, I recommend these resources as a starting point:

- Hands-On Programming with R
- R for Data Science (2e)

Here, we'll begin by becoming familiar with the ***syntax*** of R (*how we interact with the programming environment*), common R ***objects*** (*what we interact with*), and how we manipulate objects in R using ***functions*** (*performing actions on objects*).

1. The R syntax.

To begin, we'll work through some examples of how to run code in R. Below, you'll find code blocks that you can run in your R environment. Some of these code blocks are empty – that's intentional! You can fill in the blanks to practice the concepts covered in this workshop. Each code block will begin with a string like, `{r, ...}`, which tells R that the code within the block should be treated as R code. *You do not need to interact with this line!*

You can run the code blocks by clicking the green “Run” button that appears when you hover over the block, or by using keyboard shortcuts.

Some important keyboard shortcuts:

- CTRL+ENTER (Windows) or CMD+RETURN (Mac): *Run current line*
- CTRL+SHIFT+ENTER (Windows) or CMD+SHIFT+RETURN (Mac): *Run current block*

Within the code blocks, you'll frequently see the `#` character. `#` tells R (as well as most other common programming languages) to ignore the rest of the line. This is a common way to add *comments* to your code – describing what the code is doing, what the output means, or as guidance for your own workflow. For example, lets use the following intuitive code block to demonstrate how comments work:

```
1 + 1 # Add 1 and 1 together
```

```
[1] 2
```

When running the code block above, you'll see that the output is 2 and the comment is ignored. Without the `#` character, the code block above will produce an error. We will use comments throughout this workshop to describe what the code is doing or provide instruction – feel free to add your own comments too!

1.1 Operators

1.1.1 Arithmetic, comparisons, creating vectors, and storing data. Here, we'll practice giving commands to R using common mathematical operators. A *command* is a line of code that tells R to perform some action on *objects*. Let's apply some arithmetic *operators* to some *integers* below.

Basic arithmetic. The following operators intuitively interact with integers to perform basic arithmetic:

```
10 + 5 # '+' sums
```

```
[1] 15
```

```
10 - 5 # '-' subtracts
```

```
[1] 5
```

```
10 / 5 # '/' divides
```

```
[1] 2
```

```
10 * 5 # '*' multiplies
```

```
[1] 50
```

```
10 ^ 5 # '^' exponentiate
```

```
[1] 1e+05
```

“Complex” arithmetic (follows PEMDAS). Much like in basic math, R follows the orders of operations when performing arithmetic. In general, it's best to use parenthesis to make the order clear – errors with order of operations can cause many downstream issues if not handled appropriately ahead of time (*believe me...*).

```
10 + 5 - 10 * 5 # multiplication applied first
```

```
[1] -35
```

```
10 + (5 - 10) * 5 # parenthetical applied first
```

```
[1] -15
```

In the code block above, we used *operators* (i.e., `+`, `-`, `/`, `*`, `^`) to perform basic arithmetic tasks. In addition to arithmetic, operators can be used for many other tasks. We'll look at another three other types of operators below:

1. *Comparisons* (e.g., `>`, `>=`, `==`, `!=`)
2. *Creating vectors*
3. *Storing data*

Comparisons. These return a logical value (i.e., `TRUE` or `FALSE`) based on whether the comparison is true or false.

```
10 > 5 # Greater than
```

```
[1] TRUE
```

```
10 >= 5 # Greater than or equal to
```

```
[1] TRUE
```

```
10 == 5 # Equal to
```

```
[1] FALSE
```

```
10 != 5 # Not equal to
```

```
[1] TRUE
```

Creating vectors. Using the `:` operator between two integers (e.g., `1:100` creates a vector of integers from 1 to 100, inclusive).

```
# Create a vector of all integers between (and including) 1 and 100
1:100
```

```
[1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
[19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
[37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
[55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
[73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
[91] 91 92 93 94 95 96 97 98 99 100
```

```
# Arithmetic operators can be applied to integer series
```

```
5*(1:100)
```

```
[1]  5 10 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90
[19] 95 100 105 110 115 120 125 130 135 140 145 150 155 160 165 170 175 180
[37] 185 190 195 200 205 210 215 220 225 230 235 240 245 250 255 260 265 270
[55] 275 280 285 290 295 300 305 310 315 320 325 330 335 340 345 350 355 360
[73] 365 370 375 380 385 390 395 400 405 410 415 420 425 430 435 440 445 450
[91] 455 460 465 470 475 480 485 490 495 500
```

Storing data. The `<-` operator can be used to store data/output in an object.

```
# Save 10, an integer object, to the object `example_integer`
```

```
example_integer <- 10
```

```
# Call the object `example_integer` to see its value
```

```
example_integer
```

```
[1] 10
```

```
# Multiple the value of `example_integer` by 10 and store it in `example_integer`
```

```
example_integer <- 10 * example_integer
```

```
# Call the object `example_integer` to see its new value
```

```
example_integer
```

```
[1] 100
```

In the last example, we created a **object** stored in the string variable `example_integer` using the `<-` operator. This task is fundamental to programming, as it allows you to save values and recall them later without needing to re-run the code that generated those values.

When you call an object by itself, the value of that object is often returned by printing (*note that this varies between classes of objects, but generally calling an object will either print its entirety to the screen or provide a brief summary*).

Importantly, it's recommended that you name objects intuitively, as they'll be important if you need to debug or recall specific values from your data. A few tips about naming objects:

- Use `_` in place of spaces (e.g., `example_integer`)
- Object names cannot begin with integers or operators (e.g., `1example_integer` and `!example_integer` do not work)
- Capital and lowercase letters are different (e.g., `example_integer` and `Example_integer` are two different objects)
- Be consistent with your typesetting (e.g., `example_integer` & `results_output`, `EXAMPLE_INTEGER` & `RESULTS_OUTPUT`, `ExampleInteger` & `ResultsOutput`)

1.1.2 Practice with operators. Now that we know the basics, let's practice using operators and objects with the following exercise. Below is a code block with tasks enumerated in comments. Try to fill in the blanks to complete the tasks (you can use the code above as a reference).

1. Create an *object* that stores all integers from 1 to 100 (inclusive)

```
int_vec <- 1:100
```

2. For each value in the *vector*, multiply each element by $10 + 2^4$

```
int_vec <- int_vec * (10 + 2^4)
```

3. For each value in the *vector*, divide by 10×5

```
int_vec <- int_vec / (10 * 5)
```

4. How many values are greater than or equal to 50?

```
int_vec >= 50
```

```
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[37] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[49] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[61] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[73] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[85] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[97] TRUE  TRUE  TRUE  TRUE
```

5. How many values in the object are less than 25?

```
int_vec < 25
```

```
[1] TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
[13] TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
[25] TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
[37] TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
[49] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[61] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[73] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[85] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[97] FALSE FALSE FALSE FALSE
```

1.2 Functions

Functions are a type of object that store a series of commands. Function complexity ranges from simple tasks like computing a sum (i.e., `sum()`) to more complex statistical evaluations like performing linear regression (i.e., `lm()`). You can even write your own functions to perform specific tasks (we won't cover this here, but it's a powerful tool to have in your programming toolbox).

Functions have the following syntax: `function(<arguments>)`. **Arguments** can be thought of as “options” supplied to the function; each function may take zero or more arguments. Examples of arguments include the input data (e.g., `data = 0:100`) and ignoring missing values (e.g., `na.rm = TRUE`). Let's look at an example of a couple functions below:

```
# Create a vector of integers from 1 to 100
dat <- 1:100
```

```
# Obtain the maximum value in the vector
max(dat)
```

```
[1] 100
```

```
# Obtain the minimum value in the vector
min(dat)
```

```
[1] 1
```

```
# Print a basic summary of the vector
summary(dat)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1.00	25.75	50.50	50.50	75.25	100.00

To learn more about a function, you can use the `help()` function to call the **documentation** about a function (e.g., `help(sum)`). A shorthand for this is to use the `?` operator for a function (e.g., `?sum`). The documentation for a function will often include a description of what the function does, the syntax for using the function, and examples of how to use the function.

Let's return to prompts in the 1.1.2 Practice with operators section above where we counted the number of values (stored in an object) that passed our test. Originally, we counted by eye; however, using `sum()`, we can quickly compute the number of values that pass our test. For example:

```
# Using sum() with a logical vector will add the total number of `TRUE` values:
sum(1:50 > 25)
```

```
[1] 25
```

Functions can also be nested:

```
# Nested functions with `sqrt()` and `sum()`
sum(sqrt(1:50) > 5)
```

```
[1] 25
```

Applying **functions** to data stored in objects is at the core of how biological data is analyzed in R. At this point, we've covered the following concepts and we'll continue using these tools in the upcoming lessons:

1. **Operators** (e.g., `+`, `:`, `<-`)
2. **Objects** (e.g., `example_integer`)
3. **Functions** (e.g., `sum()`)

Next, we'll highlight some common types of objects and how to manipulate them.

2. Common R objects - vectors and data frames

2.1 Vectors

We've already covered a few types of *objects* above (e.g., *integers* and *functions*), so we'll begin with a object type we've already encountered: *vectors*. *Vectors* represent one-dimensional data.

2.1.1 Creating vectors. Previously, we created a *vector* of *integers* using the `:` operator and the bounding values (e.g., `1:100`). However, *vectors* can be made of many different types of data using the `c()` function. Instead of creating a vector of *integers*, we'll create a vector of strings below:

```
# Note that the wrapped text formatting is simply for readability.
```

```
example_vec <- c("Michael", "Arjun", "Margaret",  
                "Betty", "Josh", "Rajiv",  
                "Michael")
```

```
example_vec
```

```
[1] "Michael" "Arjun"   "Margaret" "Betty"   "Josh"    "Rajiv"   "Michael"
```

Numerical *vectors* can be created with the `c()` command. However, many alternatives are available for numerical *vectors* that are not available for string *vectors*. For example, the `:` operator works on numerics, but not strings, because the order/sequence of integer is inherently computable by R.

```
# 2:20 by steps of 2
```

```
example_vec <- c(2,4,6,8,10,12,14,16,18,20)
```

```
# 2:20 by steps of 2; use ?seq to see this function's arguments
```

```
example_vec <- seq(2,20,2)
```

```
# Test whether each element of each vector is the same
```

```
c(2,4,6,8,10,12,14,16,18,20) == seq(2,20,2)
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

Using the `example_vec` created above, let's practice some common ways to work with *vectors*. Note that the examples below also apply to other types of vectors (e.g., character vectors, logical vectors).

2.1.2 Common functions for vectors. Below are three commands that are common for summarizing a vector without printing the entire object.

```
length(example_vec) # Return number of elements in vector
```

```
[1] 10
```

```
head(example_vec, n = 5) # Show first 5 elements
```

```
[1] 2 4 6 8 10
```

```
tail(example_vec, n = 5) # Show last 5 elements
```

```
[1] 12 14 16 18 20
```

```
summary(example_vec)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
2.0	6.5	11.0	11.0	15.5	20.0

2.1.3 Accessing vector data. The `[]` operator is used to access specific elements in the vector. The general syntax is `<vector_name>[<index>]`, where `<index>` is the position of the element you want to access. You can also use a logical vector to access elements that meet a certain condition.

```
example_vec[1] # Access first element
```

```
[1] 2
```

```
example_vec[6] # Access sixth element
```

```
[1] 12
```

```
example_vec[length(example_vec)] # Access last element
```

```
[1] 20
```

```
example_vec[example_vec >= 12] # Access values > 12
```

```
[1] 12 14 16 18 20
```

2.1.4 Editing vectors. Below, we'll explore commands that make changes to an existing vector object.

```
# Create an integer vector containing 0 through 100 by steps of 5
example_vec <- seq(0,100,5)
```

We'll be combining the `[]` and `<-` operators to make changes to the `example_vec` object. The `[]` operator is used to specify the index we want to change and the `<-` operator is used to assign a new value to that index.

```
# Replace the first value with 10e6
example_vec[1] <- 10e6

# Add 0 to the end of the vector
example_vec <- c(example_vec, 0)

# Replace the second value with NA
example_vec[2] <- NA

# Replace any 50 with 500
example_vec[example_vec == 50] <- 500

# Print vector
example_vec
```

```
[1] 1.0e+07      NA 1.0e+01 1.5e+01 2.0e+01 2.5e+01 3.0e+01 3.5e+01 4.0e+01
[10] 4.5e+01 5.0e+02 5.5e+01 6.0e+01 6.5e+01 7.0e+01 7.5e+01 8.0e+01 8.5e+01
[19] 9.0e+01 9.5e+01 1.0e+02 0.0e+00
```

Using the information we've learned above, what does the command below do?

```
example_vec[example_vec > 15 & example_vec <= 40 & !is.na(example_vec)]
```

```
[1] 20 25 30 35 40
```

Vectors are commonly used to in R, so being familiar with how they can be filtered, accessed, and manipulated is very powerful. Though we've only scratched the surface here, we'll be using **vectors** frequently throughout this workshop and you'll likely encounter **vectors** if you perform data analysis in R.

2.1.5 Practice with vectors. Let's try an exercise *without* focusing on a **vector** of integers. Unlike integers (e.g., 1, 2, 3, 4, ...), characters in R are specified by quotations around the string, such as "salamander". Thus, 1 and "1" are entirely different objects.

In this example, we'll specify the **functions** and **arguments** that can be used (remember, you can use `help(<function name>)` or `?<function name>` for documentation). We'll also set a seed for this exercise to

ensure that random numbers are the same each time you run this code.

```
# Set seed for reproducibility
set.seed(123)

# Preview `letters` object
head(letters)
```

```
[1] "a" "b" "c" "d" "e" "f"
```

1. Use `sample()` to create a *vector object* with the following:

- Contains 100 random samples from the pre-existing `letters` object
- Samples letters with replacement using `sample()`
- *NOTE: Store your **object** in a variable called something other than `letters`*

```
letters_sample <- sample(letters, size = 100, replace = TRUE)
head(letters_sample)
```

```
[1] "o" "s" "n" "c" "j" "r"
```

2. Using your *vector object*, perform the following tasks:

- Count each letter that appears in your first name:

```
sum(sum(letters_sample == "m"),
    sum(letters_sample == "i"),
    sum(letters_sample == "c"),
    sum(letters_sample == "h"),
    sum(letters_sample == "a"),
    sum(letters_sample == "e"),
    sum(letters_sample == "l"))
```

```
[1] 25
```

- Replace each occurrence of ‘z’ with ‘x’:

```
letters_sample[letters_sample == "z"] <- "x"
```

- Add each letter in your name to the end of your character *vector*:

```
letters_sample <- c(letters_sample, "m", "i", "c", "h", "a", "e", "l")
```

- Use `sort()` on your *object*, then print the 50th through 60th elements.

```
letters_sample_sorted <- sort(letters_sample)
letters_sample_sorted[50:60]
```

```
[1] "m" "m" "m" "n" "n" "n" "n" "n" "n" "n" "o" "o"
```

Here, we’re working with random letters of the alphabet. However, these types of procedures can also be used to filter and access, for example, specific genome loci stored in a *variant call file (VCF)*.

2.2 Data frames (and R packages)

Perhaps the most common way to collect, store, and share data is in the format of a two-dimensional “table” organized in rows (observations) and columns (variables/features). If you’re familiar with an Excel Spreadsheet, then you’re familiar with R’s *data frame objects*. For example, let’s look at a *data frame* stored in the `datasets` package.

```
head(datasets::iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

In this section, we'll focus on how to generate, access, and manipulate **data frames** - the most common format to store data in R. We'll also introduce and use **packages** (such as **datasets**) which are a collection of useful tools, instructions, and data that are purpose-built to assist users in specific analyses (e.g., phylogeny or generalized linear models). In this lesson, we'll be using the **datasets package** to access some **data frames** to practice what we've learned. In later lessons, we'll use a powerful suite of packages called **tidyverse** which helps organize and visualize data (e.g., plotting) of many different formats.

2.2.1 Packages. In the example above, we called the **iris data frame** from the **datasets package** using the `::` operator. The `::` operator allows you to access an **object** from a **package** (in the example above, we accessed a **data frame**, though the same principle applies to using **functions** from a **package**). The syntax for the `::` operator is: `<package>::<object>.*`

As you may imagine, needing to call the **package** every time we use it would quickly bloat our code. Fortunately, a **package** can be loaded into your coding environment using the `library()` **function**, allowing you to succinctly call **objects** without specifying their source (just as you had with `sum()` or `sample()`). Here are a few notes on how to load **packages**, unload **packages**, and receive help:

```
library("datasets") # load the `datasets` package into the environment
library(help = "datasets") # load the documentation for the package
detach("package:datasets") # unload the package
```

Note that once you load a library into your environment for any given R session, you don't need to load it again until you start a new R session.

2.2.2 Data frames. Now that we know about **packages**, let's return to working with **data frames**. First, let's focus on how we interact with **data frames**. Some of the syntax below will be familiar to how we interacted with **vectors**.

```
# Load library and store `iris` in `df`
library("datasets")
df <- iris

# Print the first six lines of the data frame
head(df)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

In our examples of accessing data in **vectors**, we used the `[]` operator to access specific values stored within the **vector** (e.g., `example_vec[5]` to access the 5th element). This syntax worked because a **vector** is one-dimensional.

Because a **data frame** is two-dimensional, R interprets the values inside the brackets differently. Here's how integers in brackets work for accessing values in a **data frames**:

- *Integers* without a comma will return whole columns: `df[1:3]`
- *Integers* followed by a comma will return rows: `df[1:3,]`
- *Integers* preceded by a column will extract columns as a vector: `df[,1:3]`

It follows then, to access a subset of a *data frame* you can use the following syntax: `df[<rows>,<columns>]`, such as `df[1:3,2:4]` to access rows 1:3 and columns 2:4. You can also use *vectors* to reorganize your *data frames* (e.g., `df[1:10,c(5,3)]`). As with most *objects* in R, changes to an *object* are not saved unless you write those changes to an *object*.

Another important element of *data frames* is that the columns are often associated with a character string (i.e., a *column name*) that can be used for accessing information. We'll cover this and several other common *data frame* operations below.

Viewing data. In R, most objects will print their contents when called by themselves. For example, type `df` in the console below to print the contents of the `df` object to the screen. Additionally, here are a few common functions for viewing your data frame without printing the entire object:

```
head(df) # Print first 6 lines of `df`
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

```
tail(df) # Print last 6 lines of `df`
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
145	6.7	3.3	5.7	2.5	virginica
146	6.7	3.0	5.2	2.3	virginica
147	6.3	2.5	5.0	1.9	virginica
148	6.5	3.0	5.2	2.0	virginica
149	6.2	3.4	5.4	2.3	virginica
150	5.9	3.0	5.1	1.8	virginica

```
colnames(df) # Print column names of `df`
```

```
[1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"
```

```
rownames(df) # Print row names of `df`
```

```
[1] "1" "2" "3" "4" "5" "6" "7" "8" "9" "10" "11" "12"
[13] "13" "14" "15" "16" "17" "18" "19" "20" "21" "22" "23" "24"
[25] "25" "26" "27" "28" "29" "30" "31" "32" "33" "34" "35" "36"
[37] "37" "38" "39" "40" "41" "42" "43" "44" "45" "46" "47" "48"
[49] "49" "50" "51" "52" "53" "54" "55" "56" "57" "58" "59" "60"
[61] "61" "62" "63" "64" "65" "66" "67" "68" "69" "70" "71" "72"
[73] "73" "74" "75" "76" "77" "78" "79" "80" "81" "82" "83" "84"
[85] "85" "86" "87" "88" "89" "90" "91" "92" "93" "94" "95" "96"
[97] "97" "98" "99" "100" "101" "102" "103" "104" "105" "106" "107" "108"
[109] "109" "110" "111" "112" "113" "114" "115" "116" "117" "118" "119" "120"
[121] "121" "122" "123" "124" "125" "126" "127" "128" "129" "130" "131" "132"
[133] "133" "134" "135" "136" "137" "138" "139" "140" "141" "142" "143" "144"
[145] "145" "146" "147" "148" "149" "150"
```

```
nrow(df) # Print number of rows in `df`
```

```
[1] 150
```

```
ncol(df) # Print number of columns in `df`
```

```
[1] 5
```

```
summary(df) # Print summary statistics for each column in `df`
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
Min. :4.300	Min. :2.000	Min. :1.000	Min. :0.100
1st Qu.:5.100	1st Qu.:2.800	1st Qu.:1.600	1st Qu.:0.300
Median :5.800	Median :3.000	Median :4.350	Median :1.300
Mean :5.843	Mean :3.057	Mean :3.758	Mean :1.199
3rd Qu.:6.400	3rd Qu.:3.300	3rd Qu.:5.100	3rd Qu.:1.800
Max. :7.900	Max. :4.400	Max. :6.900	Max. :2.500

Species

setosa	:50
versicolor	:50
virginica	:50

Accessing data frames. We'll be using the `head()` function throughout these examples to print the first 6 lines of output. The `head()` function is not necessary for accessing data frames, but it can be helpful for previewing data without having to print it in its entirety.

```
head(  
  df[,1] # Extract the first column  
)
```

```
[1] 5.1 4.9 4.7 4.6 5.0 5.4
```

```
head(  
  df$Sepal.Length # Extract the column named "Sepal.Length"  
)
```

```
[1] 5.1 4.9 4.7 4.6 5.0 5.4
```

```
setequal(df[,1], df$Sepal.Length) #Test whether two vectors are equal
```

```
[1] TRUE
```

```
head(  
  df[,c("Species", "Sepal.Length")] # Extract multiple columns by name  
)
```

	Species	Sepal.Length
1	setosa	5.1
2	setosa	4.9
3	setosa	4.7
4	setosa	4.6
5	setosa	5.0
6	setosa	5.4

```
head(
  df[df$Species == "virginica",] # Extract rows by testing the value of a column
)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
101	6.3	3.3	6.0	2.5	virginica
102	5.8	2.7	5.1	1.9	virginica
103	7.1	3.0	5.9	2.1	virginica
104	6.3	2.9	5.6	1.8	virginica
105	6.5	3.0	5.8	2.2	virginica
106	7.6	3.0	6.6	2.1	virginica

Editing data frames. Editing a *data frame* is conceptually similar to how we edited a *vector* above, but we have an added dimension. Below we'll look at methods to add rows and columns to an existing *data frame* object.

```
# Reset the `df` object to `iris`
df <- iris
```

Creating new observations (rows)

```
# Tail df for reference
tail(df)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
145	6.7	3.3	5.7	2.5	virginica
146	6.7	3.0	5.2	2.3	virginica
147	6.3	2.5	5.0	1.9	virginica
148	6.5	3.0	5.2	2.0	virginica
149	6.2	3.4	5.4	2.3	virginica
150	5.9	3.0	5.1	1.8	virginica

```
# Append a row
df[nrow(df)+1,] <- c(100, 100, 100, 100, NA)
```

```
# Tail df again to see new row
tail(df)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
146	6.7	3.0	5.2	2.3	virginica
147	6.3	2.5	5.0	1.9	virginica
148	6.5	3.0	5.2	2.0	virginica
149	6.2	3.4	5.4	2.3	virginica
150	5.9	3.0	5.1	1.8	virginica
151	100.0	100.0	100.0	100.0	<NA>

```
# Edit a value in the new row
df[nrow(df), "Species"] <- "setosa"
```

```
# Tail df again to see edits in new row
tail(df)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
146	6.7	3.0	5.2	2.3	virginica
147	6.3	2.5	5.0	1.9	virginica
148	6.5	3.0	5.2	2.0	virginica
149	6.2	3.4	5.4	2.3	virginica

```
150      5.9      3.0      5.1      1.8 virginica
151    100.0    100.0    100.0    100.0      setosa
```

```
# Create a new column for "collection month"
df$collection_month <- rep("June", times = nrow(df)) # Append a column (variable)

# View changes after adding column
tail(df)
```

```
      Sepal.Length Sepal.Width Petal.Length Petal.Width  Species
146          6.7         3.0         5.2         2.3 virginica
147          6.3         2.5         5.0         1.9 virginica
148          6.5         3.0         5.2         2.0 virginica
149          6.2         3.4         5.4         2.3 virginica
150          5.9         3.0         5.1         1.8 virginica
151    100.0    100.0    100.0    100.0      setosa
  collection_month
146           June
147           June
148           June
149           June
150           June
151           June
```

Code interpretation exercise. Now that we've introduced *data frames*, let's try and interpret the following code – what is it doing? What is the output? Work through this line by line.

```
# Create a new copy of the `iris` data set
df <- iris

# Create a new column called "big" and set all values to FALSE
df$big <- FALSE

# For all rows where species is "setosa" and petal length is greater than 1.6, set "big" to TRUE
df[df$Species == "setosa" & df$Petal.Length > 1.6,]$big <- TRUE

# For all rows where species is "virginica" and petal length is greater than 6.0, set "big" to TRUE
df[df$Species == "virginica" & df$Petal.Length > 6.0,]$big <- TRUE

# For all rows where species is "versicolor" and petal length is greater than 4.5, set "big" to TRUE
df[df$Species == "versicolor" & df$Petal.Length > 4.5,]$big <- TRUE

# Print the number of rows where "big" is TRUE
nrow(df[df$big == TRUE,])
```

```
[1] 29
```

As you can see, increasing the dimensions of our data from one (*vector*) to two (*data frame*) increased the complexity of working with such data. The `iris` data set as an example of a very common format used to record data - particularly if your work involves fieldwork (no pun intended). In fact, having the foresight to record data in a format similar to how you store data can save a lot of time in the future!

Creating data frames from vectors. Before we move onto the final example exercise, let's walk through how to create a *data frame* from scratch using simulated data.

Below, we'll use a few common *functions* to simulate (or randomly sample) data. We'll also use some

formatting tricks that will help with making code a bit easier to read. If you're not sure what a *function* is doing below, try running each *function* by itself in the console below (and looking at that *function's* `help()`).

```
df<-data.frame(trial = rep(x = 1:50, # `rep()` replicates values of a vector
                        each = 2),
              species = rep(x = c("a","b"),
                          times = 50),
              height = rnorm(n=100, # `rnorm()` simulates values from a normal distribution
                          mean=20,
                          sd=5),
              color = sample(x = c("blue","red","green"),
                          size = 100,
                          replace = T))
```

```
colnames(df)
```

```
[1] "trial" "species" "height" "color"
```

```
nrow(df)
```

```
[1] 100
```

```
summary(df)
```

trial	species	height	color
Min. : 1.0	Length:100	Min. : 7.526	Length:100
1st Qu.:13.0	Class :character	1st Qu.:16.814	Class :character
Median :25.5	Mode :character	Median :19.936	Mode :character
Mean :25.5		Mean :19.890	
3rd Qu.:38.0		3rd Qu.:23.024	
Max. :50.0		Max. :29.357	

While empirical data analysis (i.e., analyzing observational data) may be the more immediately relevant use-case for R, simulation of data (e.g., output from `rnorm()`) is *extremely* important for hypotheses testing and generating null expectations. We return to this concept for an in-depth look in lesson 3 of this workshop.

2.2.3 Practice with data frames. Let's close out by working through a *data frame* exercise that uses everything we've learned.

```
# Load the `airquality` dataset into a data frame object
df <- airquality
```

1. Count the rows missing *Ozone* measurements. *Hint: Use the function, `is.na()`*

```
nrow(df[is.na(df$Ozone),])
```

```
[1] 37
```

2. Remove all observations from the months of *May* and *June* (you may need to do this in multiple steps). *Hint: You can use `!` to negate a comparison.*

```
df <- df[!df$Month == 5,]
df <- df[!df$Month == 6,]
```

3. How many days was *Wind* greater than 10?

```
nrow(df[df$Wind > 10,])
```

```
[1] 40
```

4. Create a column called “observer” that contains your name.

```
df$observer <- "michael"
```

5. Print the 20th line of your data frame.

```
df[20,]
```

```
      Ozone Solar.R Wind Temp Month Day observer
81      63      220 11.5   85      7  20  michael
```

Congratulations! We’ve covered the very basics of how to code in R. We’ve only scratched the surface, but you’ve now encountered many of the core elements of what makes R such a powerful tool for data science!

Above, we covered several examples of the *operators* and *functions* used to interact with *objects*. In the next lesson, we’ll introduce the **tidyverse package** (for data tidying and plotting) and work with some basic summary statistics used in data analysis.

3. Recommended reading

- Chapters 2 in *Hands on Programming with R*
- *Chapter 2 in R for Data Science*